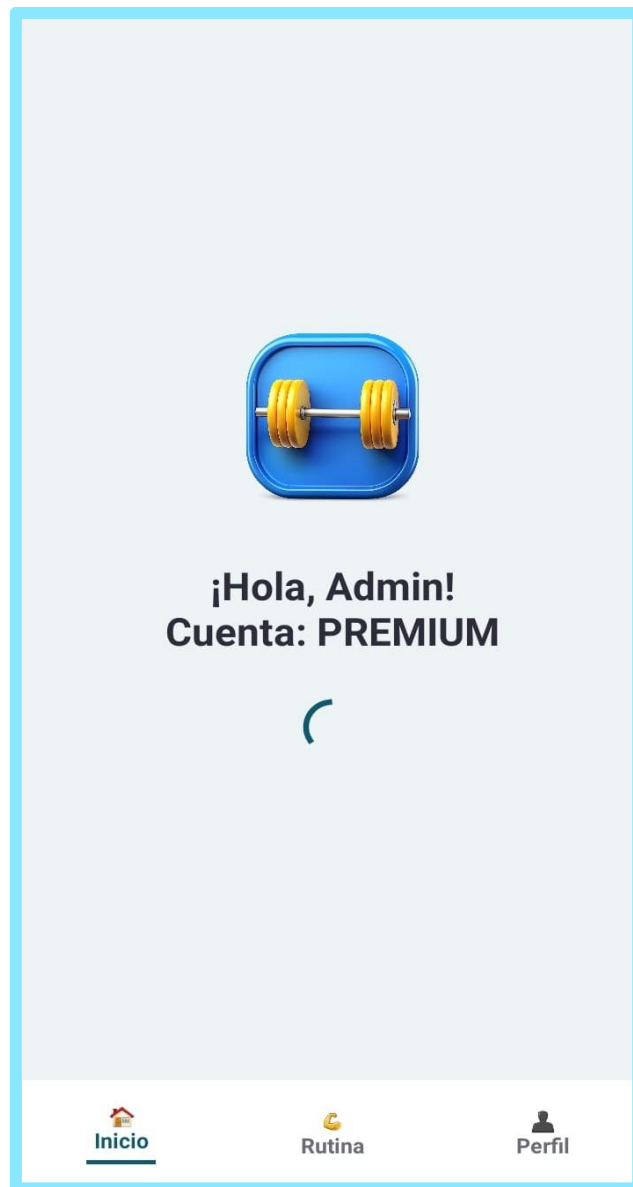




# Aplicación móvil para Android

TEMA: SEGUIMIENTO DE RUTINAS DE EJERCICIOS

**Curso:**

Aplicaciones Móviles

**Docente:**

MANAY GINES, JHOON HOMERO

**Integrantes:**

- Campos Arias Camila Nicole
  - Anyoza Joel Andre
- Yata Pinaud Belinda Adela

# Índice

1. Resumen
2. Introducción
3. Justificación
4. Necesidad que atiende la aplicación
5. Objetivos del proyecto
6. Tecnologías y herramientas utilizadas
7. Proceso de desarrollo de la aplicación
8. Arquitectura y estructura del proyecto
9. Flujo de funcionamiento de la aplicación
10. Desarrollo y funcionalidades de la aplicación
11. Conclusiones
12. Recomendaciones
13. Bibliografía

# 1. Resumen

**appRutinas** es una aplicación móvil desarrollada para Android en **Java**, cuyo objetivo es ayudar a los usuarios a organizar y dar seguimiento a sus rutinas de ejercicio de manera sencilla e intuitiva. La aplicación permite **registrarse**, **iniciar sesión**, personalizar la rutina según las características del usuario y llevar un control del progreso diario y semanal.

El sistema incorpora funciones como la selección automática de rutinas por **nivel** y **día de la semana**, reproducción de videos demostrativos mediante **VideoView**, almacenamiento local de información con **SharedPreferences**, así como una diferenciación entre usuarios **Gratuito** y **Premium**, ofreciendo funcionalidades específicas para cada tipo de cuenta.

Además, la aplicación consume una **API** mediante **Retrofit** para mostrar frases motivacionales y mejorar la experiencia del usuario. Para su desarrollo se emplearon componentes como **Spinner**, **CheckBox**, **SeekBar**, **AlertDialog**, **Toast** e **Intent**, organizando el proyecto en una estructura modular que facilita su mantenimiento.

## 2. Introducción

En la actualidad, las aplicaciones móviles se han convertido en una herramienta importante para promover hábitos saludables y facilitar el seguimiento de actividades físicas. En este contexto, **appRutinas** fue desarrollada como una aplicación para dispositivos Android que permite al usuario organizar, personalizar y controlar sus rutinas de entrenamiento de manera sencilla e intuitiva.

La aplicación integra diferentes módulos que trabajan de forma conjunta, como el **registro e inicio de sesión**, la **personalización de la rutina**, la visualización del **perfil del usuario**, el seguimiento de ejercicios diarios y un **panel principal** donde se muestra el progreso semanal, la rutina correspondiente al día y frases motivacionales obtenidas mediante una **API**. Además, incorpora contenido multimedia a través de **VideoView**, permitiendo visualizar la ejecución correcta de cada ejercicio.

Para mejorar la experiencia del usuario, la aplicación almacena información como el peso, altura, nivel de entrenamiento, objetivo y progreso mediante **SharedPreferences**, lo que permite mantener los datos disponibles durante el uso de la aplicación. Asimismo, diferencia entre usuarios **Gratuito** y **Premium**, habilitando funcionalidades específicas según el tipo de cuenta, como el acceso a niveles superiores y la gestión de ejercicios personalizados.

El proyecto fue desarrollado utilizando **Java** y **Android Studio**, organizando el código en una estructura modular compuesta por Activities, modelos, utilidades y componentes de red, facilitando la comunicación entre las distintas pantallas y el mantenimiento de la aplicación.

## 3. Justificación

El desarrollo de **appRutinas** se justifica por la necesidad de integrar la tecnología móvil en la adopción de estilos de vida saludables, optimizando la gestión del tiempo y el esfuerzo de los usuarios en sus entrenamientos diarios.

- **Importancia:** Radica en combatir la deserción en la actividad física autónoma. Muchas personas abandonan sus entrenamientos debido a la falta de organización o a la incertidumbre sobre cómo ejecutar los ejercicios. Al centralizar la rutina, las instrucciones y

el material audiovisual en una sola herramienta accesible, la aplicación se convierte en un soporte educativo y motivacional clave.

- **Beneficios:** Para el usuario: Centralización de su progreso, recordatorio automático de su rutina diaria según el día de la semana, acceso directo a guías visuales para evitar lesiones y la posibilidad de personalizar el entrenamiento según su nivel y objetivos.
- **Académicos/Técnicos:** Permite aplicar de forma práctica los conocimientos de desarrollo en Android bajo el lenguaje Java, gestionando eficazmente la persistencia de datos local, la lógica de navegación mediante Intents y la experiencia de usuario (UX/UI) en dispositivos móviles.
- **Impacto:** A mediano plazo, la aplicación busca transformar el seguimiento de ejercicios de un proceso tedioso en papel o notas dispersas, a un hábito digitalizado y automatizado. Al segmentar adecuadamente a los usuarios (gratuitos y premium), establece una base escalable para un modelo de servicio sostenible y eficiente.

## 4. Necesidad que atiende la aplicación

Una de las principales dificultades al seguir una rutina de ejercicios de forma autónoma es la falta de adherencia, provocada por no recordar qué actividad corresponde realizar cada día o perder el control del avance alcanzado.

Por esta razón, **appRutinas** atiende la necesidad de centralizar y estructurar de forma organizada la rutina diaria. Permite al usuario consultar los ejercicios programados en tiempo real, visualizar las indicaciones y videos necesarios para ejecutarlos correctamente y registrar las actividades completadas. Además, mitiga la falta de personalización en aplicaciones genéricas al segmentar el entrenamiento por el objetivo y nivel físico del usuario, facilitando un entorno seguro y constante para su desarrollo físico.

## 5. Objetivos del proyecto

### Objetivo General:

Desarrollar una aplicación móvil nativa para el sistema operativo Android que optimice la organización y el seguimiento semanal de rutinas de ejercicios, permitiendo a los usuarios interactuar con interfaces intuitivas para registrar su avance, visualizar guías demostrativas y mantener la constancia en su entrenamiento físico.

### Objetivos Específicos:

Implementar un módulo de autenticación y registro seguro con persistencia de datos local (**UserDatabase**) para salvaguardar el progreso individual de cada usuario.

Diseñar un sistema de personalización dinámico mediante componentes interactivos (Spinners y SeekBars) que adapte la dificultad del entrenamiento y restrinja los accesos según el tipo de cuenta (gratuita o premium).

Desarrollar un módulo de seguimiento semanal automatizado que detecte el día actual y renderice dinámicamente la lista de ejercicios correspondientes con sus respectivos soportes multimedia y casillas de verificación (**CheckBox**).

Garantizar la escalabilidad del proyecto aplicando buenas prácticas de estructuración de paquetes de componentes en Android Studio utilizando el lenguaje Java.

## 6. Tecnologías y herramientas utilizadas

- **Android Studio (IDE)**  
Entorno de desarrollo integrado oficial utilizado para la codificación, diseño de interfaces XML, compilación, ejecución y depuración de la aplicación.
- **Java (JDK 17+)**  
Lenguaje de programación orientado a objetos empleado para desarrollar la lógica de negocio, controladores, validaciones y funcionalidades de la aplicación.
- **Android SDK Platform**  
Conjunto de bibliotecas nativas de Android utilizado para la implementación de Activities, navegación mediante Intents y componentes de la interfaz gráfica.
- **XML**  
Lenguaje utilizado para el diseño de las interfaces de usuario, permitiendo estructurar las diferentes pantallas de la aplicación.
- **Material Components**  
Biblioteca utilizada para implementar componentes modernos como **TextInputLayout** y **TextInputEditText**, mejorando la presentación de los formularios de inicio de sesión y registro.
- **SharedPreferences**  
Mecanismo de almacenamiento local empleado para guardar información del usuario, como peso, altura, sexo, objetivo, nivel seleccionado, progreso semanal y tipo de cuenta.
- **Retrofit 2 + Gson Converter + OkHttp Logging Interceptor**  
Herramientas utilizadas para consumir la API de frases motivacionales, convertir la respuesta JSON a objetos Java y registrar las solicitudes HTTP durante el desarrollo.
- **Colecciones Java (ArrayList, HashMap y HashSet)**  
Estructuras de datos utilizadas para almacenar rutinas, organizar ejercicios por nivel y día, gestionar ejercicios personalizados y evitar registros duplicados.
- **Componentes de UI interactivos**  
Uso de **Spinner** para la selección del día y objetivos, **SeekBar** para los niveles de dificultad, **CheckBox** para registrar el progreso, **VideoView** para la reproducción de ejercicios, **ProgressBar** para mostrar el avance semanal y **AlertDialog** para confirmar acciones dentro de la aplicación.
- **Intents y Bundles**  
Mecanismo nativo de Android empleado para la navegación entre pantallas y la transferencia de información entre Activities, como el nombre y tipo de usuario.
- **Git & GitHub**  
Sistema de control de versiones distribuido utilizado para el trabajo colaborativo del equipo y el respaldo del código fuente.

## 7. Proceso de desarrollo de la aplicación

### 7.1. Fase de Análisis y Requerimientos

En esta etapa inicial se definieron las necesidades del usuario y el alcance del sistema. Se determinó que la aplicación debía funcionar de forma autónoma y ligera, optando por una arquitectura nativa en Java y un sistema de simulación de datos en memoria RAM (UserDatabase) para agilizar las pruebas de consistencia lógica sin depender de conectividad externa.

Asimismo, se establecieron las principales funcionalidades de la aplicación, como el registro e inicio de sesión, la personalización de rutinas, el seguimiento del progreso diario y semanal, la diferenciación entre usuarios Gratuito y Premium, así como la incorporación de contenido multimedia y frases motivacionales para mejorar la experiencia del usuario.

### 7.2. Fase de Diseño de Arquitectura y UI

Se diseñó la estructura de paquetes (activity, model, utils) observada en la sección anterior para asegurar la mantenibilidad del código. En paralelo, se crearon los archivos de maquetación XML en la carpeta layout, definiendo una navegación fluida mediante una barra de menús compartida (barra\_navegacion.xml).

Durante esta etapa también se diseñaron interfaces reutilizables, como el componente item\_ejercicio.xml para la visualización dinámica de cada ejercicio y la barra de navegación compartida mediante la etiqueta <include>, evitando la duplicación de código en las diferentes pantallas de la aplicación.

### 7.3. Fase de Codificación por Módulos (Iteraciones)

La construcción del código fuente se ejecutó de manera secuencial, priorizando la seguridad de acceso y luego la lógica de negocio:

#### • Iteración 1: Módulo de Autenticación y Control de Acceso (LoginActivity y RegisterActivity)

Se implementaron las interfaces de captura de credenciales y la lógica de validación de campos obligatorios. Los datos capturados en el registro se vinculan dinámicamente a la lista estática en RAM para permitir accesos inmediatos durante la misma sesión de ejecución.

Se incorporó el almacenamiento del tipo de usuario mediante SharedPreferences, permitiendo mantener la sesión activa y controlar posteriormente las funcionalidades disponibles para usuarios Gratuito y Premium.

#### • Iteración 2: Módulo de Control de Mando (HomeActivity)

Se desarrolló el tablero principal (Dashboard) encargado de recuperar la fecha del sistema, estructurar los accesos directos y servir de puente de navegación hacia las características avanzadas.

Se implementó una pantalla de bienvenida con carga temporal mediante Handler, el cálculo automático del progreso semanal, la visualización de la rutina correspondiente al día y el consumo de una API mediante Retrofit para mostrar frases motivacionales obtenidas desde un servicio web.

#### • Iteración 3: Módulo de Perfil y Reglas de Negocio (PerfilActivity y PersonalizarActivity)

Se codificaron los componentes interactivos (Spinners y SeekBars). Aquí se integró la lógica condicional que evalúa el tipo de usuario (Gratis o Premium) para restringir o liberar los niveles avanzados de entrenamiento.

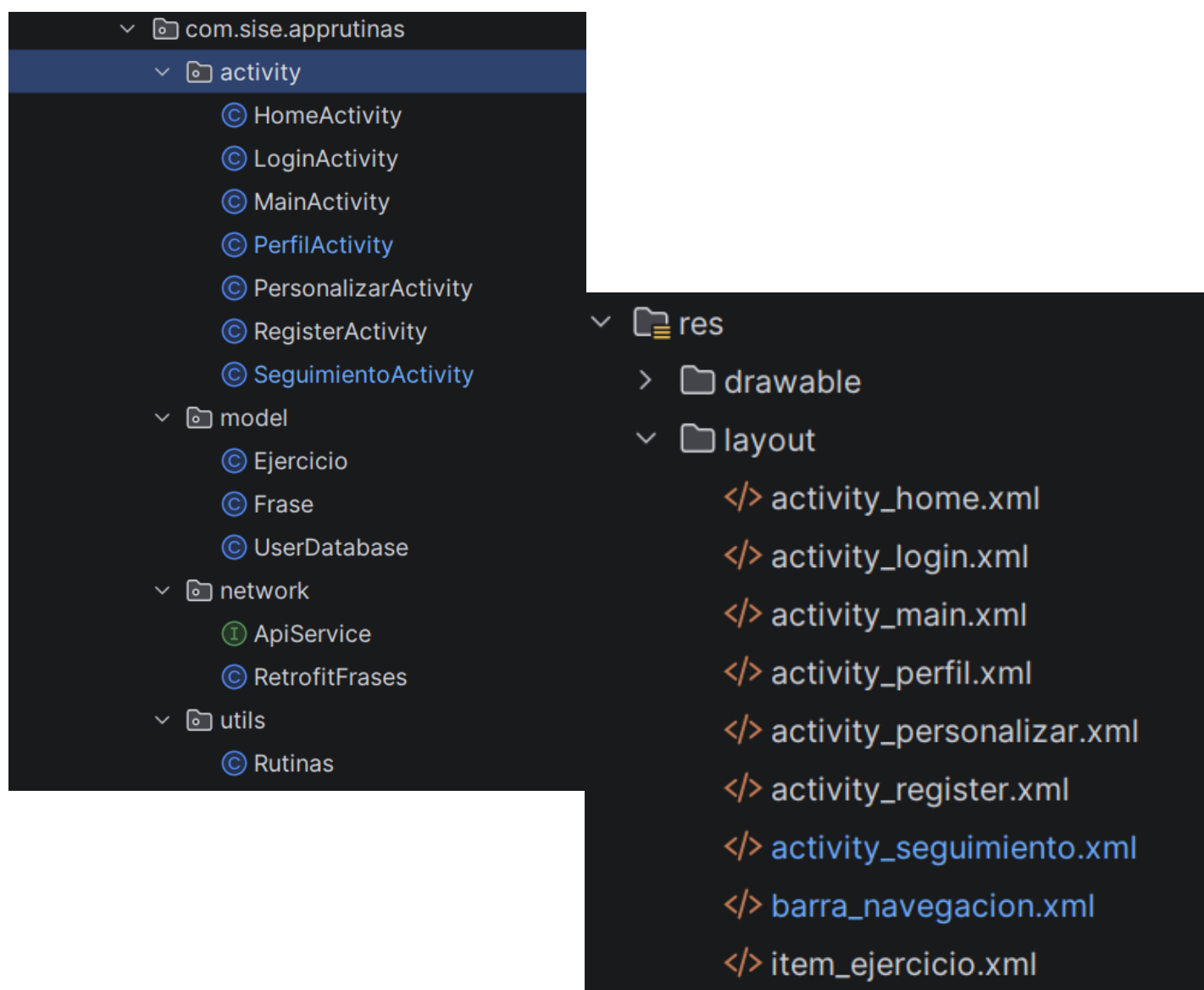
Asimismo, se desarrolló el formulario de personalización para registrar datos como peso, altura, sexo, objetivo y nivel de entrenamiento. Esta información es almacenada localmente mediante SharedPreferences para ser utilizada posteriormente por los diferentes módulos de la aplicación.

#### • Iteración 4: Módulo de Renderizado Semanal (SeguimientoActivity)

Se programó el selector de días y el adaptador que lee la clase utilitaria Rutinas.java, encargada de inflar dinámicamente el contenedor item\_ejercicio.xml con los datos de tiempo, consejos y el estado de completado mediante CheckBoxes.

Se implementó la reproducción automática de videos demostrativos mediante VideoView, la gestión de ejercicios personalizados para usuarios Premium, el cálculo del progreso diario, la actualización automática del estado de las rutinas y la persistencia del avance del usuario mediante SharedPreferences. También se incorporó un buscador de ejercicios con filtrado dinámico para facilitar la selección y personalización de las rutinas.

## 8. Arquitectura y estructura del proyecto



## 9. Flujo de funcionamiento de la aplicación

### Autenticación (Control de Acceso):

Al iniciar la aplicación (**LoginActivity**), el sistema solicita las credenciales del usuario. Estas son validadas mediante la colección estática de la clase **UserDatabase**, verificando el correo electrónico y la contraseña antes de permitir el acceso al sistema. Si el usuario aún no posee una cuenta, puede registrarse desde **RegisterActivity**, donde automáticamente se crea una cuenta de tipo **GRATUITO**.

### Segmentación de experiencia:

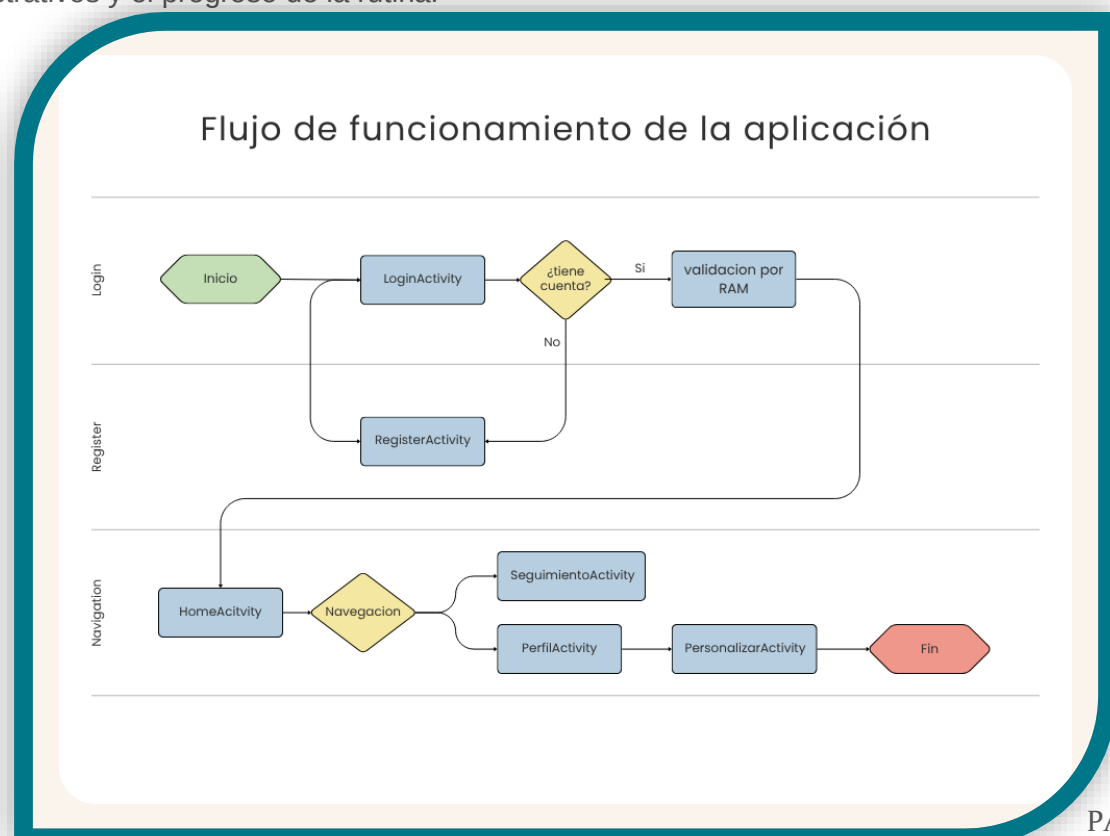
- Si el usuario inicia sesión con una cuenta **PREMIUM**, la aplicación habilita todas las funcionalidades disponibles, incluyendo el acceso a niveles avanzados y la posibilidad de agregar ejercicios personalizados a las rutinas.
- Si el usuario inicia sesión con una cuenta **GRATUITA**, determinadas funciones permanecen restringidas mediante validaciones lógicas, como los niveles superiores de entrenamiento y la gestión de ejercicios personalizados.

### Panel principal (Home):

Después de un inicio de sesión exitoso, el usuario es dirigido a **HomeActivity**, donde inicialmente se muestra una pantalla de bienvenida antes de cargar el panel principal. En esta pantalla se presenta la fecha actual, el resumen de la rutina del día, el progreso semanal, una frase motivacional obtenida mediante una API y la barra de navegación hacia los demás módulos.

### Personalización y seguimiento:

Desde el módulo **Perfil**, el usuario puede acceder a **PersonalizarActivity** para registrar datos como peso, altura, sexo, objetivo y nivel de entrenamiento. Posteriormente, en **SeguimientoActivity**, la aplicación obtiene la rutina correspondiente al nivel y al día seleccionado desde la clase utilitaria **Rutinas.java**, mostrando dinámicamente los ejercicios, videos demostrativos y el progreso de la rutina.





## 10. Desarrollo y funcionalidades de la aplicación

### 10.1. Módulo de Autenticación y Registro

#### • Pantalla de Acceso (LoginActivity)

Es la actividad lanzadora del sistema. Presenta una interfaz limpia con campos para el ingreso de correo electrónico y contraseña. Implementa controles lógicos nativos para evitar el procesamiento de campos vacíos o con formatos incorrectos. Al presionar el botón de ingreso, evalúa las credenciales contra la colección estática de UserDatabase.

Una vez autenticado el usuario, la aplicación almacena el tipo de cuenta mediante SharedPreferences y transfiere la información necesaria al módulo principal utilizando Intents, permitiendo mantener la sesión activa durante la navegación.

#### • Pantalla de Registro (RegisterActivity)

Permite dar de alta a nuevos usuarios de manera local durante el ciclo de vida de la aplicación. Solicita datos obligatorios como nombre completo, correo electrónico y contraseña. Una vez superadas las validaciones de consistencia, los datos se instancian y se añaden dinámicamente al ArrayList global, retornando el flujo de navegación al Login para permitir el acceso.

Los nuevos registros se crean inicialmente con el rol de usuario Gratuito, habilitando el acceso inmediato a las funciones básicas de la aplicación.

#### • Gestión de Credenciales (UserDatabase)

Al no depender de un motor de base de datos externo, la persistencia se simula mediante una clase contenedora estática que precarga en memoria RAM dos perfiles de prueba con roles diferenciados: un entorno de pruebas Premium (premium@test.com) y un entorno de pruebas Gratuito (gratuito@test.com).

Este mecanismo facilita las pruebas de autenticación y la validación de las funcionalidades restringidas para cada tipo de usuario.

### 10.2. Menú principal (HomeActivity)

#### • Despliegue de Información Básica

Al iniciar, la actividad recupera programáticamente la fecha actual del sistema para contextualizar al usuario en su jornada de entrenamiento.

Además, muestra un saludo personalizado, la rutina correspondiente al día, el progreso semanal del usuario y una frase motivacional obtenida mediante una API externa. Antes de mostrar el contenido principal, se presenta una pantalla de bienvenida con una carga temporal para mejorar la experiencia visual.

#### • Interfaz de Control

Dispone de un botón de acción principal para iniciar directamente el entrenamiento del día y despliega un listado histórico o resumen de los días en los que el usuario completó sus rutinas.

Cuando el día corresponde a descanso, el sistema detecta automáticamente esta condición, modifica el mensaje mostrado al usuario y deshabilita el inicio del entrenamiento.

- **Navegación Global (barra\_navegacion.xml)**

Incorpora un menú inferior persistente que gestiona los cambios de contexto del usuario hacia los módulos de perfil o seguimiento de forma fluida.

La barra de navegación fue implementada como un componente reutilizable mediante la etiqueta <include>, permitiendo mantener la misma interfaz en las diferentes pantallas y evitando duplicar código XML.

### **10.3. Módulo de Configuración y Reglas de Negocio (PerfilActivity y PersonalizarActivity)**

- **Visualización del Perfil (PerfilActivity)**

Muestra el estado actual del usuario, recuperando de forma local los últimos datos guardados como peso, altura e indicador de cuenta (Gratis o Premium).

También permite acceder al módulo de personalización y ofrece la opción de cerrar sesión, eliminando los datos almacenados de la sesión actual para regresar al Login.

- **Formulario de Personalización (PersonalizarActivity)**

Pantalla donde se capturan las métricas físicas y objetivos del atleta. Utiliza un componente Spinner para seleccionar metas específicas (como ganancia muscular, pérdida de peso o mantenimiento) y un control deslizable SeekBar para cuantificar el nivel de dificultad deseado.

Antes de aplicar los cambios, la aplicación presenta un resumen mediante un AlertDialog para que el usuario confirme la información registrada y posteriormente almacena los datos localmente utilizando SharedPreferences.

- **Control de Restricciones (Gratis vs Premium)**

Al procesar los datos, el sistema ejecuta una validación condicional. Si el usuario cuenta con el rol de pruebas gratis, la aplicación bloquea mediante código los niveles avanzados del SeekBar. Si detecta el rol premium, se libera todo el catálogo de intensidades y rutinas avanzadas.

Asimismo, las restricciones también se aplican al módulo de seguimiento, donde únicamente los usuarios Premium pueden agregar y administrar ejercicios personalizados.

### **10.4. Módulo de Seguimiento Semanal (SeguimientoActivity)**

- **Detección Automatizada del Día**

La actividad identifica el día calendario de la semana y realiza una consulta a la clase utilitaria Rutinas.java para extraer únicamente los bloques de ejercicios correspondientes a esa fecha. El usuario puede forzar la consulta de otros días de la semana a través de un componente Spinner interactivo en la cabecera.

Cada rutina se adapta al nivel previamente seleccionado por el usuario durante la personalización, mostrando únicamente los ejercicios correspondientes a esa categoría.

- **Renderizado de Ejercicios (item\_ejercicio.xml)**

Utiliza un contenedor que infla de forma secuencial una lista de tarjetas. Cada fila despliega el nombre del ejercicio, el tiempo sugerido, un consejo técnico de ejecución y un reproductor de soporte multimedia para visualizar la técnica correcta.

La reproducción de los videos se realiza automáticamente y selecciona el recurso correspondiente según el sexo registrado por el usuario durante la configuración de su perfil.

- **Control de Progreso y Personalización Práctica**

Cada ítem incluye un componente CheckBox que permite al usuario marcar en tiempo real las actividades completadas para llevar el control de su avance. Asimismo, la interfaz dispone de una opción al final del listado para que el usuario pueda añadir dinámicamente un ejercicio personalizado extra si desea complementar su entrenamiento del día.

El estado de cada ejercicio y el progreso alcanzado se almacenan localmente, permitiendo conservar la información entre las distintas pantallas de la aplicación. En el caso de los ejercicios personalizados, el usuario también puede eliminarlos posteriormente desde la misma rutina cuando dispone de una cuenta Premium.

## 11. Conclusiones

- **Cumplimiento de Objetivos Técnicos:**

Se logró desarrollar con éxito la aplicación nativa **appRutinas** para el sistema operativo Android utilizando el lenguaje Java. La integración de componentes interactivos como Spinners, SeekBars y CheckBoxes permitió consolidar interfaces de usuario intuitivas y completamente funcionales que cumplen con las expectativas de usabilidad del curso.

- **Eficiencia en el Control de Reglas de Negocio:**

La implementación de lógica condicional basada en roles de usuario demostró que es posible realizar una segmentación efectiva entre perfiles Gratuitos y Premium a nivel de código de interfaz, controlando el acceso a características avanzadas (como los niveles de intensidad física) de forma óptima.

- **Simulación de Persistencia Local:**

El diseño de la clase **UserDatabase** mediante colecciones estáticas (**ArrayList**) en memoria RAM cumplió de manera eficiente su propósito como simulador de persistencia volátil. Esto facilitó un entorno controlado y ágil para la ejecución de pruebas lógicas y validaciones de credenciales durante la fase de desarrollo, sin añadir la sobrecarga de conectividad de un motor de base de datos externo.

- **Importancia de la Arquitectura de Software:**

La refactorización del proyecto para agrupar las clases dentro del subpaquete especializado **activity** demostró que una correcta organización de directorios y recursos no solo optimiza la legibilidad y el mantenimiento del código fuente, sino que previene fallos críticos de compilación y asegura la escalabilidad de la solución técnica.

- **Experiencia de Usuario y Funcionalidad Integral:**

La integración de funcionalidades como el seguimiento semanal de rutinas, la personalización del entrenamiento, la reproducción de videos demostrativos, el almacenamiento local mediante **SharedPreferences** y el consumo de una **API** para mostrar frases motivacionales permitió ofrecer una aplicación funcional, organizada y orientada a mejorar la experiencia del usuario durante el desarrollo de sus entrenamientos.

## 12. Recomendaciones

- **Migración a Persistencia de Datos Local Robusta:**

Para futuras iteraciones comerciales o académicas del proyecto, se recomienda sustituir el almacenamiento volátil en RAM por una solución de persistencia local definitiva. Se sugiere la implementación del framework *Room* o el uso nativo de *SQLite*, lo que garantizará que los datos de registro, peso, altura y progreso diario de los usuarios no se pierdan al cerrar el ciclo de vida de la aplicación.

- **Integración de Servicios en la Nube:**

Con el objetivo de escalar la aplicación a un entorno multiusuario real, es altamente recomendable implementar servicios *Backend as a Service* (BaaS) como *Firebase*. Esto permitiría gestionar la autenticación mediante servidores seguros (Firebase Authentication), almacenar las rutinas en una base de datos NoSQL en tiempo real (Firestore) y centralizar el consumo de los videos multimedia desde la nube en lugar del directorio **raw** local.

- **Migración Arquitectónica hacia Patrones Modernos:**

Se sugiere reestructurar la lógica de negocio migrando del acoplamiento actual en las Activities hacia patrones de arquitectura de software recomendados por la industria, tales como MVVM (Model-View-ViewModel). Esto separará por completo la interfaz gráfica de la lógica de datos, facilitando la implementación de pruebas unitarias automatizadas.

- **Optimización del Manejo Multimedia:**

Para mejorar el rendimiento del dispositivo y evitar el consumo excesivo de memoria caché al reproducir los videos de los ejercicios, se recomienda la integración de librerías avanzadas de manejo de medios, como *Jetpack Media3* o *ExoPlayer*, en reemplazo de los componentes nativos estándar.

- **Incorporación de Funcionalidades de Seguimiento Avanzado:**

Como mejora futura, se recomienda incorporar un sistema de **estadísticas e historial de entrenamiento**, permitiendo al usuario visualizar gráficos de progreso, registrar metas alcanzadas y consultar el desempeño obtenido durante diferentes periodos de tiempo. Esto complementaría el seguimiento diario y semanal ya implementado en la aplicación.

## 13. Bibliografía

- Deitel, P., Deitel, H., & Deitel, A. (2015). Android para Programadores: Una aproximación basada en componentes de la app (2da ed.). Prentice Hall.
- Google Developers. (2024). Documentación oficial de Android: Guía para desarrolladores de aplicaciones nativas en Java. Android Developer Portal.  
<https://developer.android.com/guides>
- Schildt, H. (2019). Java: The Complete Reference (11va ed.). McGraw-Hill Education.
- Sierra, K., & Bates, B. (2020). Head First Android Development: A Brain-Friendly Guide (2da ed.). O'Reilly Media.